

My Ultimate Cheatsheet for Conventional Commit Types: Simplified for Everyday Use

B bavaga.com/blog/2025/01/27/my-ultimate-conventional-commit-types-cheatsheet

Wilhelm Haas

January 27, 2025

I first learned about [Conventional Commits](#) in 2020, while managing a small platform engineering project. We were searching for a simple way to generate release notes automatically, and that's when we stumbled upon conventional commits. The developers loved it immediately, so we decided to integrate it into our daily workflow.

At the time, I wasn't too involved in commits, commit messages, or their structure. My focus was more on exploring new features in tools like Google Sheets . However, since 2023, I've been back in the software engineering field, and that's when I had to dive into the world of commit messages, especially conventional commits.

Conventional commits provide a structured standard for writing commit messages concisely. While I find the general approach very useful, I've always struggled with identifying the correct commit type for certain commits. Types like **feat** and **fix** are straightforward, but what about dependency updates? Or simple style changes? These grey areas always gave me headaches.

To simplify things for myself (and maybe for you), I decided to create a personal cheatsheet. After doing some research and reviewing [repositories listed on the conventional commits site](#) , I compiled the most commonly used commit types and their respective use cases. Sit back, relax, and enjoy .

Analyzing Commit Patterns Across Repositories

As previously mentioned, I analyzed some [repositories listed on the Conventional Commits site](#) to explore which and how commit types are utilized in real-world projects. The goal was to identify which commit types are used most often.

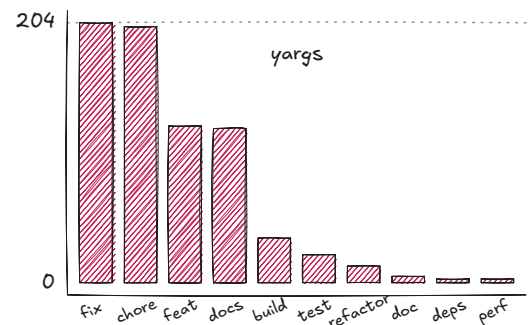
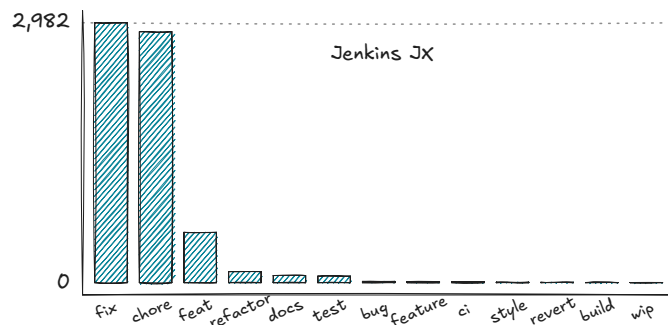
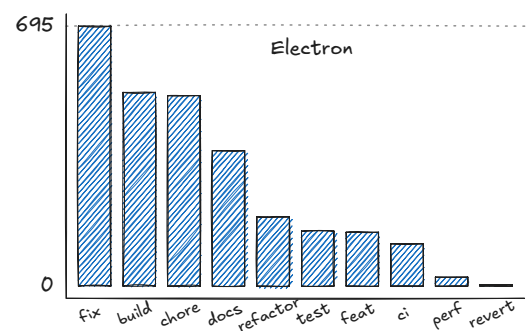
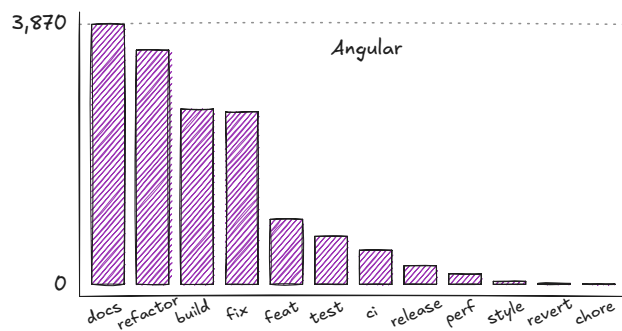
To achieve this, I parsed commit messages from the **main** branches of these repositories and created a histogram to visualize the distribution of commit types.

Before diving into the data, I sanitized the commit messages to ensure accuracy and consistency:

- Commit messages that didn't adhere to the Conventional Commit format were excluded.

- Commit types with only a single occurrence or made before adopting Conventional Commits were omitted.

The histograms reveal some interesting patterns.



For example, [Angular](#) shows a strong emphasis on **docs** commits, reflecting the importance of comprehensive documentation. On the other hand, [Electron](#) sees a higher number of **fix** and **build** commits, while [Jenkins JX](#) and [yargs](#) relies heavily on **fix** and **chore**.

Based on my personal experience and this analysis, I've distilled the 11 most essential commit types: **feat**, **fix**, **chore**, **perf**, **docs**, **build**, **revert**, **style**, **refactor**, **ci** and **test**.

These commit types form the foundation of my cheatsheet, which you'll find in the next section. It's designed to help you and me choose the right type for every commit and make the messages clearer and more effective.

Commit Types Cheatsheet

feat – New Feature or Functionality

feat is used for adding new features or functionality.

When to Use

- Adding a brand-new feature

- Implementing new functionality that wasn't there before
- Introducing a new behavior (e.g., a new UI component)
- Adding a new configuration option or parameter to an existing system

Examples

- `feat: provide google sheets adapter`
- `feat(ui): add dark mode to the user interface`
- `feat(api): add support for pagination in user endpoint`

fix – Fix a Bug

`fix` is used to address actual bugs that cause incorrect behavior in production code.

When to Use

- Correcting unintentional or erroneous behavior
- Fixing a known bug
- Correcting invalid or malfunctioning visual styles (e.g., fixing incorrect color values)
- Resolving crashes or runtime errors

Examples

- `fix: correct CSS color for button background`
- `fix: null pointer handling`
- `fix(frontend): remove flickering effect on page refresh`

perf – Performance Improvements

`perf` is used for changes that bring measurable performance improvements.

When to Use

- Optimizing existing code for speed or memory usage
- Reducing overhead in database queries or API calls
- Improving rendering or load times
- Adding caching mechanisms to frequently accessed data

Examples

- `perf: reduce number of redundant API calls`
- `perf(ui): improve table rendering performance`
- `perf: add caching for user session data`

refactor – Code Refactoring without Changing Behavior

refactor is for structural improvements to the code without changing its behavior. Unlike **style**, **refactor** focuses on enhancing the internal structure, logic, or organization of the code without altering its external behavior.

When to Use

- Restructuring or reorganizing code for clarity, maintainability, or scalability
- Extracting common logic into utility functions
- Splitting large modules into smaller, more focused ones
- Making a code section fail-safe without changing functionality
- Adding **Optional** to handle potential null values gracefully
- Simplifying complex conditions or loops
- Adding a **@Nullable** annotation to indicate that a value can be null, helping static analysis tools warn developers and reduce the risk of **NullPointerExceptions**.
- Renaming variables or methods for readability and clarity

Examples

- **refactor: extract utility functions for data validation**
- **refactor(ui): separate styling logic from component logic**
- **refactor: use Optional to handle potential null values in data processing**
- **refactor: simplify nested loops**
- **refactor: rename variable temp to temperature**

style – Code Formatting and Style-Only Changes

style is for cosmetic changes to code that do not affect its behavior. Key Difference from **refactor**: **style** changes are purely superficial and do not affect the structure, semantics, or functionality of the code.

When to Use

- Formatting the codebase (e.g., via Prettier, ESLint)
- Adjusting whitespace, indentation, line breaks, or punctuation
- Making purely cosmetic changes that do **not** affect behavior

Examples

- **style: reformat code with ESLint rules**
- **style: change indentation from 2 to 4 spaces**
- **style: fix formatting inconsistencies across multiple files**

test – Tests and Test-Related Changes

`test` is used for adding, modifying, fixing or improving tests.

When to Use

- Adding or updating unit, integration, or end-to-end tests
- Refactoring test code or changing test data
- Fixing broken test scripts
- Adding quality assurance scripts
- Covering edge cases in existing tests

Examples

- `test: add integration tests for checkout process`
- `test(auth): improve token validation tests`
- `test: add load testing script for performance checks`
- `test: cover edge cases for user registration validation`

docs – Documentation

`docs` is used for changes to documentation, comments, or API descriptions.

When to Use

- Making changes to documentation files (e.g., `README.md`, `CHANGELOG.md`)
- Adding or updating inline code comments, docstrings, or usage guides
- Writing API documentation
- Updating setup instructions

Examples

- `docs: update README to include installation steps`
- `docs: add comments to public methods`
- `docs: document API usage examples for new endpoints`

build – Build Process or Dependencies

`build` should be used for changes that impact the build process or production dependencies, including tools and configurations necessary for application deployment or runtime.

When to Use

- Updating build scripts (Webpack, Rollup, etc.)

- Changing or upgrading dependencies (npm, Maven, Gradle, etc.) that affect production code
- Modifying how the application is bundled or deployed
- Adjusting Docker or Kubernetes configurations

Examples

- `build: upgrade webpack to version 5`
- `build(deps): update express to v4.18.1` ← (Dependency for production code)
- `build: update Dockerfile for multi-stage builds`

ci – Continuous Integration

`ci` is used for changes to CI/CD configurations or workflows.

When to Use

- Changing CI/CD configuration files or scripts
- Updating workflows for GitHub Actions, GitLab CI, Jenkins, etc.
- Adding new CI steps (e.g., code coverage, automated security scans)

Examples

- `ci: add code quality checks in GitHub Actions`
- `ci: configure Jenkins pipeline for integration tests`
- `ci: add security scan step in GitLab pipeline`

chore – Maintenance and Routine Tasks

`chore` is used for administrative or supportive tasks that do not impact production code.

When to Use

- Miscellaneous tasks or project administration that doesn't fit other types
- Updating `.gitignore`, adjusting package scripts, or other project settings
- Renaming or moving files/folders without changing actual code logic
- Adjusting or creating scripts that help with local/manual testing or make development easier
- Adding `@SuppressWarnings` annotations to resolve build or IDE warnings
- Updating development dependencies (e.g., `eslint`, `jest`)

Examples

- `chore: update .gitignore to exclude .idea files`
- `chore: reorganize folder structure for better clarity`
- `chore: adjust local test script for manual QA`

- `chore: suppress unchecked cast warnings in legacy code`
- `chore(deps): update eslint to v8.14.0`

revert – Revert a Previous Commit

`revert` is used to roll back a previous commit.

When to Use

- Undoing or rolling back a previous commit when necessary
- Typically references the commit hash that is being reverted

Examples

- `revert: "feat: add social login feature"`
- `revert: "fix: correct CSS color for button background"`

My last words for today

I started using this cheatsheet in my daily workflow. I'm not a master yet, but it has already reduced my cognitive load when creating commit messages. Hopefully, it will do the same for you. Happy committing!